Software Architecture

Hand-in Assignment

Week 10

Instructions:

- Submit your solutions through GitHub Classroom and **remember** to submit a link to your repository in BrightSpace. Otherwise, I will have a hard time distinguishing between the different groups.
- In this assignment you should add images/references to your drawings in the Git Repository.

Description

During this hand-in you will practice the following:

- Analyze cyclomatic complexity of a program by hand
- Implement unit tests that ensure full coverage based on this analysis

Cyclomatic Complexity

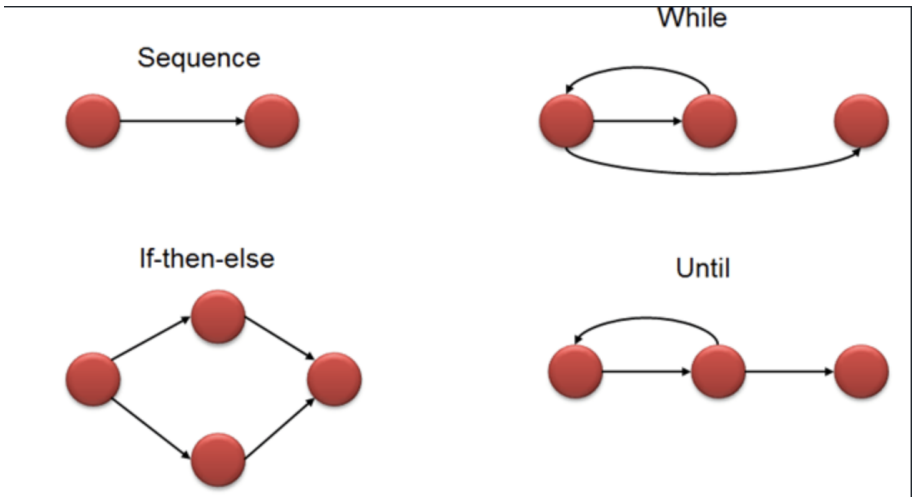
Cyclomatic Complexity is a metric that can be used as a proxy for the complexity of a program. Specifically, the metric measures the number of independent paths through a piece of software, i.e. how many different combinations of unique actions are made possible by the conditional statements of a program.

The concept of cyclomatic complexity is closely tied to the concept of a *program control graph* which is a way to represent the structure of a computer program using directed graphs. Each node of the graph represents a block of code that is executed sequentially, such as assignment statements or performing arithmetic. Nodes corresponding to conditionals cause the graph to split into different paths.

Conditionals in C/C++ include:

- `if`, `if else`, and `else`
- `while`
- `do while`
- `switch`

A short summary of the different constructs can be seen in the image below:



Common Confusion Points

- `if` statements always have an implicit `else`

```
if (sum > threshold)
{
    sum = threshold
}
else
{
    // do nothing
}

// is equivalent to
if (sum > threshold)
{
    sum = threshold
}
// else is omitted since it does nothing
```

- The edge from the terminal to the start node should *not* be counted in the V(g) formula. This is introduced when counting cycles.
- Compound statements
- V(g) can be calculated for different portions of a program, like a single function. Traditionally, you would not count the complexity of other functions being called.

References

The following material may be useful:

- docs/mccabe.pdf
- Summary of rules.
- Additional examples.

Exercise

For drawing, I recommend using drawio.

For the following functions defined in `snippets.hpp`:

- `min`
- `SortedVector::insert`
- `SortedVector::contains`
- `SortedVector::binary_search`
- `SortedVector::binary_insert`

Do the following:

1. Draw the control graph.
2. Calculate V(g) using the formula.
3. Calculate V(g) by listing the number of independent cyclic paths.
4. Save the image as `[name of function].png` and **upload** in your Git Repo.

Ensure that each image clearly indicates the calculations.